

FaceCoach — Build Brief

Build me a full-stack web app called **FaceCoach** — an AI-powered real-time public speaking coach that runs entirely in the browser, with no backend server required for the core AI features.

PROJECT VISION

People lose job interviews, fail presentations, and bomb pitches not because their content is bad, but because of HOW they deliver it — poor eye contact, nervous filler words, talking too fast or too slow, awkward silences. A human speaking coach costs money and isn't always available right before a big moment. FaceCoach solves this by giving anyone free, instant, AI-powered speech coaching using nothing but their own webcam and browser.

The entire core experience — face tracking and speech analysis — runs client-side using TensorFlow.js and the Web Speech API, meaning zero server cost for the heaviest part of the app. Only the final, deep AI feedback step calls Gemini once per session. This makes the app effectively free to run at any scale, and trivially easy for anyone to clone, deploy, and use.

This is an OPEN SOURCE project. Treat every architectural decision as if a stranger will clone this repo tomorrow with zero context, add only a Gemini API key, and expect it to run.

TECH STACK (with reasoning)

- **React (Vite) + TypeScript** — Vite for fast dev server and build times, TypeScript for type safety across all the metric calculation logic (this app has a lot of numeric scoring math that benefits from strict typing)
- **TensorFlow.js** (`@tensorflow-models/face-landmarks-detection`) — runs the face mesh model directly in the browser using WebGL, no Python, no server inference. Detects facial landmarks ~468 points, used to derive head pose / gaze direction
- **Web Speech API** (`SpeechRecognition` / `webkitSpeechRecognition`) — browser-native, completely free, no library install. Used for live transcription, filler word detection, and words-per-minute calculation
- **Gemini API** (`gemini-2.5-flash` model) — called exactly ONCE at the end of a session, through a serverless function, to generate qualitative human-style coaching feedback from the session transcript and metrics
- **Chart.js** (via `react-chartjs-2`) — renders the score-over-time line chart on the report page and the progress-over-sessions chart on the history page
- **localStorage** — the entire persistence layer. No database, no Supabase, no backend state. All session data lives in the user's own browser
- **Tailwind CSS** — utility-first styling, dark theme, consistent spacing/sizing tokens
- **React Router** — client-side routing between the 4 pages
- **Deploy target: Vercel** — chosen because serverless functions (for the Gemini proxy) deploy automatically alongside the static frontend, and the free tier comfortably covers this app's usage pattern

NON-GOALS (explicitly out of scope for v1)

- No user accounts / authentication

- No cloud database of any kind
- No mobile app — web only, but should be responsive
- No multi-language support — English only for filler word detection in v1
- No video recording/storage — only live analysis, the video itself is never saved anywhere

PAGES / ROUTES

1. `/` → Landing page with hero, features list, "Start Session" CTA button
2. `/session` → Live coaching session page
3. `/report/:id` → Post-session report page
4. `/history` → Past sessions list from localStorage

PAGE 1 — LANDING (`/`)

Purpose: communicate what the app does in under 5 seconds, build trust ("this is free, this is private, this actually works"), and get the user into a session with minimal friction.

- Dark theme, minimal, lots of breathing room — this should NOT look like a cluttered SaaS landing page
- Sticky top nav: logo/wordmark "FaceCoach" on left, "History" link and "Start Session" button on right
- Hero section:
 - Small eyebrow badge above the headline: "🔒 100% Free · Runs in Your Browser"
 - Headline: "Your AI Public Speaking Coach"
 - Subtext: "Real-time eye contact, filler word, and pace tracking. Powered by AI. Runs entirely in your browser. Nothing you say or do leaves your device until you ask for feedback."
 - Primary CTA button, large, orange: "Start Session →"
 - Small reassurance line beneath the button: "No sign-up. No installs. Your camera never leaves your browser."
- Feature section, 3 cards in a row (stack vertically on mobile):
 1. 🧐 **Eye Contact Tracking** — "AI watches whether you're engaging the camera or looking away, the same way a real audience would notice"
 2. 🗣️ **Filler Word Detection** — "Catches every 'um', 'like', and 'you know' in real time using live speech transcription"
 3. 📝 **AI Deep Feedback** — "After your session, Gemini analyzes your full transcript and gives you specific, encouraging coaching notes"
- "How it works" mini-section, 3 numbered steps with short descriptions: 1) Turn on your camera 2) Speak naturally for as long as you want 3) Get instant metrics plus AI feedback
- Footer: small text — "Open source on GitHub" with a link, plus "Built with React, TensorFlow.js, and Gemini"

PAGE 2 — LIVE SESSION (`/session`)

This is the core page of the entire app. Get this right above all else.

Layout

Split layout, full viewport height:

LEFT SIDE (60% width):

- Live webcam feed using `getUserMedia({ video: true, audio: true })`
- Render the video feed into a `<video>` element, muted, autoplay
- Overlay a `<canvas>` element exactly positioned on top of the video to draw face detection boxes/landmarks
- TensorFlow.js `face-landmarks-detection` model loaded once on page mount (show a loading spinner "Loading AI model..." until ready — this can take a few seconds on first load)
- Run detection in a loop every 200ms using `requestAnimationFrame` combined with a timestamp check (don't run detection every single frame, that's wasteful — throttle to ~5 times per second)
- From the landmarks, derive whether the user is facing the camera: calculate the horizontal offset between the nose tip landmark and the midpoint between the two eyes. If this offset exceeds a threshold (tune this empirically, start around 15% of face width), classify as "looking away"
- Draw a bounding box around the detected face: green if "looking at camera", yellow/orange if "looking away", red if no face detected at all
- If no face detected for more than 2 continuous seconds, show a prominent on-screen warning: "⚠️ No face detected — make sure you're visible"

RIGHT SIDE (40% width) — LIVE DASHBOARD:

Four live metric cards, each updating at least once per second, arranged in a 2x2 grid:

1. 👁️ Eye Contact %

- Calculated as: (seconds where "looking at camera" = true) / (total elapsed seconds) × 100
- Color the number green if >70%, yellow if 40-70%, red if <40%

2. 🗣️ Filler Words

- Running count, incremented every time a filler word is detected in the live transcript
- List of words to detect (case-insensitive, word-boundary matched, not substring matched): "um", "uh", "umm", "uuh", "like", "basically", "actually", "you know", "so", "right", "kind of", "sort of", "literally", "i mean"
- Be careful with word-boundary matching — "like" should not match inside "likely" or "unlike"

3. 🕒 Speaking Pace

- Calculated as a rolling words-per-minute over the last 15 seconds of speech (not the whole session average — pace should feel "live" and responsive to how the user is speaking RIGHT NOW)
- Display the number with a small label: "Good" (120-160 wpm), "Slow down a bit" (<100 wpm), "Too fast" (>180 wpm), or just the range label that fits

4. 🛑 Long Pauses

- Count of distinct silences longer than 3 seconds while the session is active
- Detected via the Web Speech API's gap between `onresult` events, or via a separate audio-level/volume check using the Web Audio API `AnalyserNode` if you want more precision (prefer the simpler speech-event-gap approach for v1)

Live Alert Ticker

Below the 4 cards, a small scrolling/fading ticker area that shows the most recent 1-2 alerts, each visible for ~3 seconds before fading:

- "👁️ You looked away" — fires when eye contact lost continuously for >2 sec
- "🗣️ Filler word: 'umm'" — fires on each new filler word detected
- "⚡ Speaking too fast!" — fires when rolling wpm exceeds 180
- "🐢 Speaking too slow" — fires when rolling wpm drops below 90
- "🔊 Long pause detected" — fires when a silence crosses the 3-second threshold

Top of Page

- A live session timer (MM:SS) counting up from the moment the user clicks "Start"
- A "Start Session" button that requests webcam/mic permissions and begins all tracking
- Before clicking Start, show a calm pre-session state: webcam preview only, no metrics yet, with a hint: "Click Start when you're ready to begin your speech"

Bottom of Page

- "End Session" button — stops all tracking, calculates final metrics, saves the session object to localStorage, then navigates to `/report/:id` for the newly created session
- Confirm before ending if the session is very short (<10 seconds), to avoid accidental clicks: "Session is very short, are you sure you want to end?"

Web Speech API Implementation Details

- Use `SpeechRecognition` with fallback to `webkitSpeechRecognition` for browser compat (mainly Chrome/Edge support this well; Safari/Firefox support is partial — add a note in the UI if the browser doesn't support it: "Speech detection isn't supported in this browser, please use Chrome or Edge")
- Configuration: `continuous = true`, `interimResults = true`, `lang = 'en-US'`
- On `onresult`: append the final transcript segments to a running transcript string, run filler-word detection against the newly finalized segment only (not the whole transcript each time, for performance), and update the rolling word count used for wpm calculation
- On `onerror` / `onend`: if the recognition engine stops unexpectedly (it sometimes auto-stops after ~60 seconds of silence in some browsers), automatically restart it as long as the session is still active
- Store a timestamped event log array throughout the session — every filler word, every pause, every "looked away" moment gets pushed as `{ time: secondsFromStart, type: 'filler' | 'pause' | 'lookaway' | 'fast' | 'slow', detail: string }`. This array becomes the event timeline shown later on the report page

PAGE 3 — REPORT (`/report/:id`)

Loaded by reading the session with matching `id` out of the `facecoach_sessions` array in localStorage. If the id isn't found (e.g. user manually typed a bad URL, or cleared their browser data), show a friendly "Session not found" state with a link back to history/home — don't crash.

Top of page

- Large Overall Score (0–100), shown as a big number with a circular progress ring or similar visual treatment

- Score color: green if 80+, yellow if 50-79, red if below 50
- Calculation:

```
score = (eyeContact% * 0.4) + (paceScore * 0.3) + (fillerPenalty * 0.3)
fillerPenalty = max(0, 100 - fillerWords * 5)
paceScore = 100 if 120-160wpm, decreases outside range
```

For `paceScore`, implement a smooth falloff rather than a hard cliff: at exactly 120-160 wpm score is 100, and it linearly decreases by some reasonable rate the further outside that range the average pace falls, floored at 0.

Four metric summary cards

Same four metrics as the live session (final values for the whole session): Eye Contact %, Filler Word Count, Average WPM, Long Pause Count. Each card should also show a one-line qualitative label, e.g. "Eye Contact: 84% — Excellent" or "Filler Words: 12 — Could improve."

Score timeline chart

Line chart (Chart.js) — X axis is time in seconds through the session, Y axis is a rolling combined score (recompute the same scoring formula using a trailing 15-second window of data, so the line shows how performance fluctuated minute to minute, not just one flat number). This is one of the most visually impressive parts of the report, so make sure it renders smoothly and has clear axis labels.

Event timeline list

A scrollable list of every event from the session's `events` array, in chronological order, each formatted like:

- 0:23 — 🗣️ Filler word: "umm"
- 0:45 — 👁️ Eye contact lost for 4 seconds
- 1:12 — ⚡ Speaking pace: 190 wpm (too fast)
- 1:40 — ⏸️ Long pause (5 seconds)

If there are more than ~20 events, this list should be scrollable within a fixed-height container rather than pushing the whole page down indefinitely.

AI Feedback section

- On page load (if this session doesn't already have cached AI feedback saved), automatically call the `/api/analyze` serverless function once with the transcript + final metrics
- Show a loading skeleton/spinner while waiting: "Gemini is analyzing your session..."
- Exact prompt to send to Gemini:

```
"You are an expert public speaking coach. The user just finished a speaking session. Transcript: [transcript].
Metrics: Eye contact [x]%, Filler words: [n], Avg pace: [wpm] wpm, Long pauses: [n]. Give exactly 3 strengths and
3 specific improvements in simple, encouraging language. Format as JSON: {strengths: [], improvements: []}"
```

- Parse the JSON response defensively (Gemini sometimes wraps JSON in markdown code fences — strip those before parsing, and wrap the parse in a try/catch with a sensible fallback message if it fails)
- Display the 3 strengths in green-tinted cards with a ✅ icon, and the 3 improvements in orange-tinted cards with a 💡 icon

- Once received, cache this feedback into the session object in localStorage so reopening this report later doesn't re-call Gemini unnecessarily

Bottom actions

- "Save to History" — if not already saved (it likely auto-saves when the session ends, so this can instead be "✓ Saved to History" as a passive confirmation rather than an action)
- "Start New Session" — navigates back to `/session`

PAGE 4 — HISTORY (`/history`)

- Read the full `facecoach_sessions` array from localStorage, sorted newest first
- If empty, show a friendly empty state: "No sessions yet — start your first one to begin tracking progress" with a "Start Session" button
- Otherwise, show a grid/list of session cards, each showing: date + time, overall score (color-coded same as report page), duration (MM:SS), and the single weakest metric for that session as a small tag (e.g. "Needs work: Filler words")
- Clicking any card navigates to `/report/:id` for that session
- Above the list, a progress chart (Chart.js line chart): X axis = session number or date, Y axis = overall score, so the user can visually see if they're improving session over session
- Optional but nice: a small stats summary row above the chart — "Total sessions: 7", "Best score: 91", "Average score: 68"
- A small "Clear All History" button (with a confirmation dialog) for users who want to reset, since this is just localStorage and easy to clear

LOCALSTORAGE SCHEMA

Key: `facecoach_sessions` → JSON array of session objects:

```
{
  "id": "uuid",
  "created_at": "ISO timestamp",
  "duration_seconds": 0,
  "overall_score": 0,
  "eye_contact_percent": 0,
  "filler_word_count": 0,
  "avg_wpm": 0,
  "long_pause_count": 0,
  "transcript": "string",
  "ai_strengths": [],
  "ai_improvements": [],
  "events": [{ "time": "", "type": "", "detail": "" }]
}
```

IMPORTANT CONSTRAINTS

- TensorFlow.js must run CLIENT SIDE only — no server, no Python, no GPU server dependency of any kind

- Web Speech API runs client side only — no server
- The Gemini API call must go through a serverless API route (e.g. `/api/analyze.ts` if using Vercel's serverless functions) so the API key is never exposed to the browser — never call the Gemini API directly from client-side code
- NO Supabase, NO Postgres, NO database of any kind — `localStorage` is the only persistence layer for this version
- No Docker, no containers, no backend server process to manage
- Everything must work within Vercel's free tier limits (serverless function execution time, bandwidth, etc.)
- Use a single environment variable: `GEMINI_API_KEY` — set in Vercel's project settings for deployment, and in `.env.local` for local dev
- This is an OPEN SOURCE project on GitHub. Anyone who clones the repo should be able to run it locally with ONLY these steps: clone repo, `npm install`, copy `.env.example` to `.env.local` and paste in their own free Gemini API key, `npm run dev`. No other setup, no other accounts, no other services required
- Handle the case where `localStorage` is unavailable or full gracefully (e.g. private browsing modes in some browsers restrict it) — wrap `localStorage` reads/writes in try/catch and show a non-blocking warning if it fails, rather than crashing the app
- Handle the case where the browser doesn't support `SpeechRecognition` or the TensorFlow.js model fails to load (e.g. very old browser, no WebGL) — degrade gracefully with a clear message rather than a blank screen or silent failure

UI DESIGN

- Dark background: `#0a0a0a`
- Card background: `#111111`
- Accent color: `#f97316` (orange) — used for primary buttons, active states, and key highlights
- Green for good metrics: `#22c55e`
- Red for bad metrics: `#ef4444`
- Yellow/amber for "okay/middle" metrics: `#eab308`
- Font: Inter (load via Google Fonts or self-host)
- Rounded cards (`rounded-xl` or similar), subtle border: `border-white/10`, soft shadow on hover for interactive cards
- Smooth number animations on the live dashboard metrics — when a number updates, it should count/transition smoothly rather than jump instantly, to feel polished
- Consistent spacing scale throughout (Tailwind's default spacing scale is fine, just be consistent)
- All interactive elements need visible hover and focus states for accessibility
- Responsive: the live session split-layout can stack vertically on small screens (webcam on top, dashboard below) rather than side by side

README REQUIREMENTS

The generated README.md must include, in this order:

1. A one-paragraph description of what FaceCoach does and why it's useful
2. A short list of the tech stack used

3. A "Getting Started" section with exact numbered steps: clone the repo, install dependencies, get a free Gemini API key (link to <https://aistudio.google.com>), create `.env.local` from `.env.example`, run the dev server, open the local URL
4. A note clarifying that all session data is stored only in the user's own browser via `localStorage` — nothing is sent anywhere except the one-time Gemini API call for feedback, and explicitly state that the webcam video itself is never uploaded or stored anywhere
5. A short "Deploying to Vercel" section
6. A short "Contributing" section inviting PRs, since this is meant to be open source
7. A license section (suggest MIT license as the default, generate a `LICENSE` file accordingly)

BUILD ORDER

1. Set up Vite + React + TypeScript + Tailwind + React Router, confirm a blank page runs locally first
2. Install all dependencies (`@tensorflow-models/face-landmarks-detection`, `@tensorflow/tfjs`, `chart.js`, `react-chartjs-2`, `uuid` for generating session ids)
3. Build the landing page first (no logic needed, pure UI) — confirms styling/theme is working before anything complex
4. Build the session page with ONLY the webcam feed working (no AI yet) — confirms camera permissions and video rendering work
5. Add TensorFlow.js face detection on top of the working webcam feed — confirms the model loads and landmarks are detected, before wiring up any scoring logic
6. Add Web Speech API transcription, `console.log` the live transcript first to confirm it's capturing speech correctly before building the filler-word/pace logic on top
7. Wire up all 4 live metric calculations and the alert ticker
8. Implement the "End Session" flow: assemble the full session object, save it into the `facecoach_sessions` array in `localStorage`, generate a `uuid`, navigate to `/report/:id`
9. Build the report page UI and the score calculation logic using only the saved metrics (no Gemini yet) — confirms the math and charts work with real session data
10. Add the Gemini API integration via the serverless `/api/analyze` route, wire it into the report page's AI Feedback section
11. Build the history page, reading from the same `localStorage` array
12. Write the full README per the requirements above, generate `.env.example` and `LICENSE`
13. Final polish pass: responsive check, loading states, error states, then deploy to Vercel and verify the deployed version works end-to-end with a real Gemini key set in Vercel's environment variables